

DOCUMENTACIÓN TÉCNICA

Laboratorio IaaS — Despliegue de Infraestructura con Incus

Keycloak · FastAPI · CockroachDB · Frontend React

Presentado Por:

Camilo Delgado Ramirez · Tomas Montoya Buitrago

Michael Naranjo Chito

Sistemas Operativos — Tecnología en Desarrollo de Software

1. Introducción

Este documento describe la implementación completa de un laboratorio IaaS basado en Incus. Se trabaja con contenedores de sistema LXC (no privilegiados), lo que impone restricciones específicas que esta guía respeta en su totalidad.

Restricciones del entorno que aplican en toda la guía:

- apt-get y apt NO deben usarse dentro de los contenedores LXC. Todo software se descarga en la VM Host y se transfiere con incus file push.
- systemctl y systemd no funcionan en contenedores LXC no privilegiados. Los servicios se gestionan con nohup ... & y disown -a. El reinicio se hace con incus restart <contenedor>.
- El systemctl SÍ funciona en la VM Host. Solo está prohibido dentro de los contenedores LXC.
- Python3, tar, bash y herramientas POSIX básicas sí están disponibles en las imágenes Ubuntu base sin necesidad de apt.

2. Objetivos del Laboratorio

- Implementar una infraestructura IaaS ligera usando Incus como hipervisor de contenedores.
- Implementar autenticación centralizada con Keycloak (JWT, JWKS).
- Desplegar un backend API usando FastAPI con validación JWT sin consulta centralizada.
- Construir un clúster distribuido de tres nodos con CockroachDB conectado de manera horizontal

3. Arquitectura General del Sistema

La VM Host Ubuntu ejecuta Nginx como proxy y el daemon de Incus. Dentro de Incus corren seis contenedores de sistema conectados por el bridge incusbr0:

Contenedor	Servicio	Puerto	Rol
frontend-node	React + serve	3000	Interfaz de usuario
backend-api	FastAPI + Uvicorn	8000	API CRUD protegida con JWT
keycloak-server	Keycloak 24.x	8080	Autenticación SSO, emisión JWT
db-node1	CockroachDB	26257/8081	Nodo DB distribuido 1
db-node2	CockroachDB	26257/8082	Nodo DB distribuido 2
db-node3	CockroachDB	26257/8083	Nodo DB distribuido 3

3.1 Topología de Red

Incus crea automáticamente el bridge incusbr0. Todos los contenedores reciben IPs en el segmento privado. Nginx en el host actúa como único punto de entrada desde el exterior.

```
➜ incus list -c n,s,4
```

NAME	STATE	IPV4
auth-node1	RUNNING	10.172.148.156 (eth0)
backend-api	RUNNING	10.172.148.175 (eth0)
db-node1	RUNNING	10.172.148.116 (eth0)
db-node2	RUNNING	10.172.148.207 (eth0)
db-node3	RUNNING	10.172.148.114 (eth0)
frontend-node	RUNNING	10.172.148.193 (eth0)

3.2 Flujo de Peticiones

- El usuario abre el navegador → Nginx (host:80) → frontend-node:3000.
- El frontend redirige al usuario a Keycloak (host:8080) → keycloak-server:8080.
- Keycloak autentica y emite un token JWT firmado con RS256.
- El frontend almacena el JWT y lo adjunta como Bearer en cada petición a /api/*.
- Nginx redirige /api/ → backend-api:8000. FastAPI valida el JWT con la clave pública JWKS de Keycloak (sin consulta centralizada por petición).
- FastAPI ejecuta la operación SQL contra el clúster CockroachDB y devuelve JSON.

4. Requisitos del Sistema

Recurso	Recomendado
RAM	8 GB (mínimo 6 GB)
CPU	4 núcleos
Disco	60 GB SSD
SO Host	Ubuntu 22.04 LTS o 24.04 LTS
Arquitectura	ARM64 o x86_64

5. Instalación e Inicialización de Incus

```
# En la VM Host
sudo apt update && sudo apt install incus -y
sudo incus admin init
```

Respuestas recomendadas en el asistente de init:

- Storage backend: btrfs (o dir si btrfs no está disponible).
- Network bridge: incusbr0, NAT habilitado, DHCP habilitado.
- Resto de opciones: valores por defecto.

5.1 Verificación de Red

```
incus network list
```

```
→ ~ incus network list
```

NAME	TYPE	MANAGED	IPV4	IPV6	DESCRIPTION	USED BY	STATE
docker0	bridge	NO				0	
enp0s1	physical	NO				0	
incusbr0	bridge	YES	10.172.148.1/24	fd42:a564:48a0:86a3::1/64		7	CREATED
lo	loopback	NO				0	

```
ip addr show incusbr0
```

```
→ ~ ip addr show incusbr0
```

```
3: incusbr0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 10:66:6a:1c:9c:e1 brd ff:ff:ff:ff:ff:ff
    inet 10.172.148.1/24 brd 10.172.148.255 scope global incusbr0
        valid_lft forever preferred_lft forever
    inet6 fd42:a564:48a0:86a3::1/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::1266:6aff:fe1c:9ce1/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

```
incus list -c n,s,4
```

```
→ ~ incus list -c n,s,4
```

NAME	STATE	IPV4
auth-node1	RUNNING	10.172.148.156 (eth0)
backend-api	RUNNING	10.172.148.175 (eth0)
db-node1	RUNNING	10.172.148.116 (eth0)
db-node2	RUNNING	10.172.148.207 (eth0)
db-node3	RUNNING	10.172.148.114 (eth0)
frontend-node	RUNNING	10.172.148.193 (eth0)

6. Creación de Contenedores y Asignación de Recursos

6.1 Lanzar Contenedores

```
incus launch images:ubuntu/22.04 frontend-node
incus launch images:ubuntu/22.04 backend-api
incus launch images:ubuntu/22.04 keycloak-server
incus launch images:ubuntu/22.04 db-node1
incus launch images:ubuntu/22.04 db-node2
incus launch images:ubuntu/22.04 db-node3

# Verificar estado y anotar IPs
incus list
```

6.2 Asignación de Recursos

```
# Keycloak necesita más recursos
incus config set keycloak-server limits.memory 2GB
incus config set keycloak-server limits.cpu 2

# Backend API
incus config set backend-api limits.memory 512MB
incus config set backend-api limits.cpu 1

# Nodos CockroachDB
incus config set db-node1 limits.memory 1GB
incus config set db-node2 limits.memory 1GB
incus config set db-node3 limits.memory 1GB
```

7. Configuración de Nginx (VM Host)

```
# En la VM Host
sudo apt install nginx -y
sudo nano /etc/nginx/sites-available/keycloak
```

7.1 Archivo de Configuración

Editar /etc/nginx/sites-available/default con las IPs reales de los contenedores:

```
server {
    listen 80;
    listen [::]:80;
    server_name 192.168.64.2;
    #Frontend
    location / {
        proxy_pass http://10.172.148.193:3000;
        proxy_set_header Host $host;
```

```
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
#FastAPI Backend
location /api/ {
    proxy_pass http://10.172.148.175:8000/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
server {
    listen 8080;
    listen [::]:8080;
    server_name 192.168.64.2;
    #Keycloak
    location / {
        proxy_pass http://10.172.148.156:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme; # (Limpiado el duplicado aqui )
        proxy_set_header X-Forwarded-Port 8080;
        proxy_set_header X-Frame-Options "SAMEORIGIN";
        proxy_buffer_size 128k;
        proxy_buffers 4 256k;
        proxy_busy_buffers_size 256k;
    }
}
```

7.2 Aplicar Configuración

```
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl status nginx
```

- Verificar con: `sudo nginx -t` — debe responder 'syntax is ok' y 'test is successful'.

8. Despliegue del Frontend React

8.1 Build en Máquina de Desarrollo

```
# En tu PC de desarrollo (no en el host ni en el contenedor)
npm run build
tar -czvf frontend-build.tar.gz dist/
```

8.2 Transferencia a la VM Host y al Contenedor

```
# Desde tu PC de desarrollo, copiar a la VM Host
scp frontend-build.tar.gz usuario_vm@IP_VM:/tmp/

# Desde la VM Host, inyectar al contenedor
incus file push /tmp/frontend-build.tar.gz frontend-node/opt/
```

8.3 Despliegue dentro del Contenedor

```
incus exec frontend-node -- bash
cd /opt mkdir -p dist && cd dist
tar -xzf ../frontend-build.tar.gz
cd /opt/dist/
(python3 -m http.server 3000 > /opt/frontend.log 2>&1 &)
disown -a
exit
```

8.4 Verificación

```
incus exec frontend-node -- ss -tlnp | grep 3000
[→ ~ incus exec frontend-node -- ss -tlnp | grep 3000
LISTEN 0      5            0.0.0.0:3000    0.0.0.0:*
→ ~ □
```

8.5 Reinicio del Frontend

```
reiniciar el contenedor completo
incus restart frontend-node
# Luego volver a lanzar el servidor (ver comandos anteriores)
```

9. Despliegue del Backend FastAPI

9.1 Preparar Dependencias en la VM Host

Primero corregir el reloj si aparece el error 'Release file is not valid yet':

```
# En la VM Host (solo si hay error de fecha)
sudo timedatectl set-ntp false
sudo timedatectl set-time "2026-05-29 12:00:00"
date # verificar
```

9.2 Descarga de Dependencias Python

```
# En la VM Host
mkdir -p /tmp/dependencias && cd /tmp

pip install --target=/tmp/dependencias \
  --upgrade --force-reinstall \
  --python-version 3.12 \
  --only-binary=:all: \
  fastapi uvicorn pydantic pydantic_core \
  python-keycloak python-jose[cryptography] \
  requests psycpg2-binary \
  --break-system-packages

# Empaquetar todo
tar -czvf /tmp/backend-empaquetado.tar.gz -C /tmp/dependencias .
```

9.3 Transferir e Instalar en el Contenedor

```
# Transferir dependencias y código al contenedor
incus file push /tmp/backend-empaquetado.tar.gz backend-node/opt/
incus file push main.py backend-node/opt/

# Desplegar dentro del contenedor
incus exec backend-api -- bash -c "
  cd /opt
  rm -rf deps
  mkdir -p deps
  tar -xzf backend-empaquetado.tar.gz -C deps
  cp main.py deps/
"
```

9.4 Arranque de Uvicorn

```
incus exec backend-api -- bash
cd /opt
(python3 -m uvicorn main:app --host 0.0.0.0 --port 8000 > /opt/backend.log 2>&1 &)
disown -a
ss -tlnp | grep 8000

# Verificar que todo funciona correctamente

ss -tlnp | grep 8000
LISTEN 0      2048          0.0.0.0:8000      0.0.0.0:*
```

9.6 Reinicio del Backend


```
# desde la vm
incus restart backend-api
# Luego volver a lanzar el servidor (ver comandos anteriores)
```

10. Configuración de Keycloak

10.1 Instalar Java (sin apt en el contenedor)

```
# En la VM Host: descargar OpenJDK 21 como tarball
wget
https://download.java.net/java/GA/jdk21/fd2272bbf8e04c3dbae13770090416c/35/GPL/openjdk-21_linux-x64_bin.tar.gz
wget
https://github.com/keycloak/keycloak/releases/download/24.0.5/keycloak-24.0.5.tar.gz

# Crear directorio en el contenedor y transferir
incus exec keycloak-server -- mkdir -p /opt/java/
incus file push openjdk-21_linux-x64_bin.tar.gz keycloak-server/opt/java/
incus exec keycloak-server -- tar -xzf /opt/java/openjdk-21_linux-x64_bin.tar.gz -C /opt/java/
```

10.2 Instalar Keycloak

```
# En la VM Host: descargar Keycloak
wget
https://github.com/keycloak/keycloak/releases/download/24.0.0/keycloak-24.0.0.tar.gz

# Transferir al contenedor
incus exec keycloak-server -- mkdir -p /opt/
incus file push keycloak-24.0.0.tar.gz keycloak-server/opt/
incus exec keycloak-server -- tar -xzf /opt/keycloak-24.0.0.tar.gz -C /opt/
```

10.3 Arranque de Keycloak

```
incus exec keycloak-server -- bash -c "
    JAVA_HOME=/opt/java/jdk-21
    PATH=$JAVA_HOME/bin:$PATH
    export JAVA_HOME PATH
    (nohup /opt/keycloak-24.0.0/bin/kc.sh start-dev \
    > /opt/keycloak.log 2>&1 &)
    disown -a
"

# Monitorear arranque (esperar 'Listening on')
incus exec keycloak-server -- bash -c "tail -f /opt/keycloak.log"
```

10.4 Configuración del Realm y Clientes

Acceder a `http://IP_VM_HOST:8080`. Usuario `admin` / contraseña definida en primer arranque.

- Crear realm: laboratorio (no usar el realm 'master' para aplicaciones).
- Crear cliente frontend: tipo Public, Authorization Code + PKCE habilitado, Valid Redirect URIs: `http://IP_VM_HOST/*`.
- Crear cliente backend-api: tipo Confidential, anotar el Client Secret para `main.py`.
- Crear usuario de prueba: `test-user`, asignar contraseña temporal.

10.5 Endpoint JWKS (Validación de Tokens)

FastAPI valida los JWT usando la clave pública de Keycloak obtenida del endpoint JWKS, sin consultar Keycloak en cada petición. Esto es lo que hace el método `get_public_key()` del `main.py`.

```
# Endpoint de claves públicas (JWKS)
http://10.172.148.156:8080/realms/laboratorio/protocol/openid-connect/certs

# Probar que el endpoint responde
curl http://10.172.148.156:8080/realms/laboratorio/protocol/openid-connect/certs
```

10.6 Reinicio de Keycloak

```
# Reiniciar el contenedor completo
incus restart keycloak-server

# Luego relanzar Keycloak
incus exec keycloak-server -- bash -c "
    JAVA_HOME=/opt/java/jdk-21
    PATH=$JAVA_HOME/bin:$PATH
    export JAVA_HOME PATH
    (nohup /opt/keycloak-24.0.0/bin/kc.sh start-dev > /opt/keycloak.log 2>&1 &)
    disown -a
"
```

11. Configuración del Clúster CockroachDB

11.1 Descarga y Distribución del Binario

```
# En la VM Host: descargar el binario correcto para tu arquitectura
# ARM64 (Apple Silicon, Raspberry Pi):
wget https://binaries.cockroachdb.com/cockroach-v23.2.5.linux-arm64.tgz

# x86_64 (Intel/AMD):
# wget https://binaries.cockroachdb.com/cockroach-v23.2.5.linux-amd64.tgz

tar -xzf cockroach-v23.2.5.linux-arm64.tgz --strip-components=1
# Ahora tienes el binario 'cockroach' en el directorio actual

# Copiar a los tres nodos
for node in db-node1 db-node2 db-node3; do
```

```
incus exec $node -- mkdir -p /usr/local/bin/ /var/lib/cockroach/
incus file push ./cockroach $node/usr/local/bin/cockroach
incus exec $node -- chmod +x /usr/local/bin/cockroach
done
```

11.2 Arranque de los Nodos (sin systemctl)

```
# Nodo 1 (ajustar IPs a las de tu entorno)
incus exec db-node1 -- bash -c "
(nohup cockroach start --insecure \
  --store=/var/lib/cockroach/node1 \
  --listen-addr=10.172.148.116:26257 \
  --http-addr=10.172.148.116:8081 \
  --join=10.172.148.116,10.172.148.207,10.172.148.114 \
  > /var/log/cockroach-node1.log 2>&1 &)
disown -a
"

# Nodo 2
incus exec db-node2 -- bash -c "
(nohup cockroach start --insecure \
  --store=/var/lib/cockroach/node2 \
  --listen-addr=10.172.148.207:26257 \
  --http-addr=10.172.148.207:8082 \
  --join=10.172.148.116,10.172.148.207,10.172.148.114 \
  > /var/log/cockroach-node2.log 2>&1 &)
disown -a
"

# Nodo 3
incus exec db-node3 -- bash -c "
(nohup cockroach start --insecure \
  --store=/var/lib/cockroach/node3 \
  --listen-addr=10.172.148.114:26257 \
  --http-addr=10.172.148.114:8083 \
  --join=10.172.148.116,10.172.148.207,10.172.148.114 \
  > /var/log/cockroach-node3.log 2>&1 &)
disown -a
"
```

11.3 Inicialización del Clúster y Base de Datos

```
# Inicializar el clúster (una sola vez, desde db-node1)
incus exec db-node1 -- cockroach init --insecure --host=10.172.148.116:26257

# Verificar estado del clúster
incus exec db-node1 -- cockroach node status --insecure --host=10.172.148.116:26257

# Crear base de datos y tabla de prueba
incus exec db-node1 -- cockroach sql --insecure --host=localhost:26257 --execute="
CREATE DATABASE IF NOT EXISTS laboratorio;
USE laboratorio;
CREATE TABLE IF NOT EXISTS items (id SERIAL PRIMARY KEY, nombre STRING);
INSERT INTO items (nombre) VALUES ('item-1'), ('item-2'), ('item-3');
SELECT * FROM items;
"
```

11.4 Reinicio de Nodos CockroachDB

```
# Reiniciar un contenedor DB
incus restart db-node1

# Volver a lanzar CockroachDB en ese nodo
incus exec db-node1 -- bash -c "
  (nohup cockroach start --insecure \
    --store=/var/lib/cockroach/node1 \
    --listen-addr=10.172.148.116:26257 \
    --http-addr=10.172.148.116:8081 \
    --join=10.172.148.116,10.172.148.207,10.172.148.114 \
    > /var/log/cockroach-node1.log 2>&1 &)
  disown -a
"
# Nota: NO ejecutar cockroach init de nuevo. El clúster ya fue inicializado.
```

12. Cold Start — Inicio Completo del Sistema

Procedimiento para reiniciar todo el entorno desde cero (p. ej. después de apagar la VM Host):

12.1 Verificar e Iniciar Contenedores

```
incus list
incus start frontend-node backend-api keycloak-server db-node1 db-node2 db-node3
```

12.2 Relanzar Servicios en los Contenedores

Después de incus start, los contenedores están corriendo pero los procesos (uvicorn, keycloak, cockroach) deben relanzarse manualmente ya que no hay systemd.

```
# 1. CockroachDB (los tres nodos)
incus exec db-node1 -- bash -c "(nohup cockroach start --insecure
--store=/var/lib/cockroach/node1 --listen-addr=10.172.148.116:26257
--http-addr=10.172.148.116:8081 --join=10.172.148.116,10.172.148.207,10.172.148.114 >
/var/log/cockroach-node1.log 2>&1 &) && disown -a"
incus exec db-node2 -- bash -c "(nohup cockroach start --insecure
--store=/var/lib/cockroach/node2 --listen-addr=10.172.148.207:26257
--http-addr=10.172.148.207:8082 --join=10.172.148.116,10.172.148.207,10.172.148.114 >
/var/log/cockroach-node2.log 2>&1 &) && disown -a"
incus exec db-node3 -- bash -c "(nohup cockroach start --insecure
--store=/var/lib/cockroach/node3 --listen-addr=10.172.148.114:26257
--http-addr=10.172.148.114:8083 --join=10.172.148.116,10.172.148.207,10.172.148.114 >
/var/log/cockroach-node3.log 2>&1 &) && disown -a"

# 2. Keycloak
incus exec keycloak-server -- bash -c "JAVA_HOME=/opt/java/jdk-21
PATH=$JAVA_HOME/bin:$PATH export JAVA_HOME PATH; (nohup /opt/keycloak-24.0.0/bin/kc.sh
start-dev > /opt/keycloak.log 2>&1 &) && disown -a"

# 3. Backend FastAPI
```

```
incus exec backend-api -- bash -c "cd /opt/deps && (export PYTHONPATH=. && python3 -m uvicorn main:app --host 0.0.0.0 --port 8000 > /opt/backend.log 2>&1 &) && disown -a"

# 4. Frontend
incus exec frontend-node -- bash -c "cd /opt/build && (serve -s . -l 3000 > /opt/frontend.log 2>&1 &) && disown -a"

# 5. Nginx (en el host, systemctl sí funciona aquí)
sudo systemctl restart nginx
```

13. Troubleshooting

13.1 Error: 'Release file is not valid yet'

El reloj del sistema está desincronizado. Esto solo afecta al host (al descargar paquetes).

```
sudo timedatectl set-ntp false
sudo timedatectl set-time "2026-05-29 12:00:00"
date
```

13.2 Error: ModuleNotFoundError pydantic_core

Binarios incompatibles de Python. Asegurarse de descargar para la versión y arquitectura correctas.

```
# Descargar forzando binarios para Python 3.12 y la arquitectura del contenedor
pip install --only-binary=:all: --python-version 3.12 <paquete>
```

13.3 Swagger muestra 'File not found /openapi.json'

Falta root_path en FastAPI cuando se usa Nginx como proxy en /api/.

```
app = FastAPI(title="IaaS Lab API", root_path="/api")
```

13.4 Proceso muere al cerrar la terminal

Usar disown -a después de lanzar el proceso con &.

```
(nohup comando > log.log 2>&1 &)
disown -a
```

13.5 CORS bloqueado en el navegador

allow_origins no puede ser ['*'] cuando allow_credentials=True. Especificar el origen exacto.

```
allow_origins=["http://IP_VM_HOST"], # No usar "*"

```

13.6 CockroachDB no forma el clúster

Verificar que los tres nodos estén corriendo antes de ejecutar cockroach init, y que las IPs en --join coincidan exactamente.

```
# Verificar que cada nodo responde
incus exec db-node1 -- cockroach node status --insecure --host=10.172.148.116:26257
# Si dice 'connection refused', el proceso no levantó - revisar el log:
incus exec db-node1 -- tail -20 /var/log/cockroach-node1.log

```

13.7 Keycloak tarda en arrancar

Keycloak en modo start-dev puede tardar 30-60 segundos. Monitorear con tail:

```
incus exec keycloak-server -- bash -c "tail -f /opt/keycloak.log"
# Esperar el mensaje: 'Listening on: http://0.0.0.0:8080'

```

14. Validaciones Finales

Prueba	Cómo verificar	Resultado esperado
Contenedores activos	incus list	Estado RUNNING en todos
Frontend visible	Abrir http://IP_VM en el navegador	Página de login del frontend
Keycloak accesible	Abrir http://IP_VM:8080	Pantalla de login de Keycloak
Swagger de la API	Abrir http://IP_VM/api/docs	Documentación interactiva de FastAPI
Login con JWT	POST /api/token con usuario de prueba	JWT generado
Ruta protegida	GET /api/items con Bearer token	200 OK con lista de items
Rechazo sin token	GET /api/items sin header	401 Unauthorized
Clúster DB	cockroach node status --insecure	3 nodos en estado Live
Resiliencia DB	incus stop db-node3 → GET /api/items	200 OK (quórum 2/3 Raft)

15. Referencia de Puertos y Accesos

Servicio	Puerto	Acceso desde	URL de ejemplo
Frontend React	80 (Nginx)	Navegador del host	http://IP_VM
FastAPI (Swagger)	80 /api (Nginx)	Navegador del host	http://IP_VM/api/docs
Keycloak Admin	8080 (Nginx)	Navegador del host	http://IP_VM:8080
CockroachDB SQL	26257	Desde backend-api	psycopg2 connection string
CockroachDB Dashboard	8081/8082/8083	Directo al contenedor	http://10.172.148.116:8081

17. Conclusión

El laboratorio implementa una arquitectura IaaS funcional y completa usando Incus como plataforma de contenedores de sistema. La solución demuestra segmentación real de servicios, redes virtuales, autenticación centralizada con JWT, APIs protegidas y bases de datos distribuidas, todo operado con despliegues manuales controlados que reflejan escenarios reales de administración de infraestructura.

Las restricciones del entorno LXC (sin systemd, sin apt en contenedores) representan una oportunidad de aprendizaje: obligan al administrador a comprender los fundamentos de cada servicio en lugar de delegarlos a gestores de paquetes o de procesos automáticos.